



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа КР-1
«Мобильно приложение, реализующее метод Карацубы»
по курсу «Численные методы линейной алгебры»

Студент группы ИУ9-71Б Лысенко В. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Цель работы

Разработать мобильное приложение на Kotlin или Dart, реализующее метод А.А. Карацубы для перемножения длинных чисел.

2 Практическая реализация

Dart-приложение содержит 2 поля ввода для целых чисел; 2 текстовые строки, содержащие двоичное представление введённых чисел; 2 текстовых поля, содержащие информацию о времени отработки алгоритма Карацубы и обычно-го перемножения чисел; а также текстовое поле, демонстрирующее результат умножения.

Код программы:

```
import 'dart:math';
import 'package:flutter/material.dart';

void main() {
  runApp(KaratsubaApp());
}

class KaratsubaApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Karatsuba',
      home: KaratsubaScreen(),
    );
  }
}

class KaratsubaScreen extends StatefulWidget {
  @override
  _KaratsubaScreenState createState() => _KaratsubaScreenState();
}

class _KaratsubaScreenState extends State<KaratsubaScreen> {
  final _uController = TextEditingController();
  final _vController = TextEditingController();

  String _binaryU = '';
```

```

String _binaryV = '';
String _normal_time = '';
String _karatsuba_time = '';
String _result = '';

String _toBinary(BigInt number) => number.toRadixString(2);

BigInt _longMultiplication(BigInt u, BigInt v) {
  List<int> uDigits = u.toString().split('').map(int.parse).toList();
  List<int> vDigits = v.toString().split('').map(int.parse).toList();

  List<int> result = List.filled(uDigits.length + vDigits.length, 0);

  for (int i = uDigits.length - 1; i >= 0; i--) {
    for (int j = vDigits.length - 1; j >= 0; j--) {
      int product = uDigits[i] * vDigits[j];
      int posLow = i + j + 1;
      int posHigh = i + j;

      product += result[posLow];
      result[posLow] = product % 10;
      result[posHigh] += product ~/ 10;
    }
  }

  BigInt finalResult = BigInt.zero;
  for (int digit in result) {
    finalResult = finalResult * BigInt.from(10) + BigInt.from(digit);
  }

  return finalResult;
}

BigInt _karatsuba(BigInt u, BigInt v) {
  if (u.bitLength <= 64 || v.bitLength <= 64) {
    return u * v;
  }

  int n = max(u.bitLength, v.bitLength);
  int m = (n ~/ 2);

  BigInt high1 = u >> m;
  BigInt low1 = u - (high1 << m);
  BigInt high2 = v >> m;
  BigInt low2 = v - (high2 << m);

```

```

        BigInt z0 = _karatsuba(low1, low2);
        BigInt z1 = _karatsuba((low1 + high1), (low2 + high2));
        BigInt z2 = _karatsuba(high1, high2);

        return (z2 << (2 * m)) + ((z1 - z2 - z0) << m) + z0;
    }

    void _calculate() {
        setState(() {
            BigInt? u = BigInt.tryParse(_uController.text);
            BigInt? v = BigInt.tryParse(_vController.text);

            if (u == null || v == null) {
                _binaryU = '';
                _binaryV = '';
                _normal_time = 'Invalid input';
                _karatsuba_time = 'Invalid input';
                _result = 'Invalid input';
                return;
            }

            _binaryU = _toBinary(u);
            _binaryV = _toBinary(v);

            final naiveStopwatch = Stopwatch()..start();
            BigInt naiveResult = _longMultiplication(u, v);
            naiveStopwatch.stop();
            _normal_time = '${naiveStopwatch.elapsedMicroseconds} mc';

            final karatsubaStopwatch = Stopwatch()..start();
            BigInt karatsubaResult = _karatsuba(u, v);
            karatsubaStopwatch.stop();
            _karatsuba_time = '${karatsubaStopwatch.elapsedMicroseconds} mc';

            _result = karatsubaResult.toString();
        });
    }

    @override
    Widget build(BuildContext context) {
        return Scaffold(
            appBar: AppBar(title: Text('Метод АА.. Карацубы')),
            body: Center(
                child: Padding(
                    padding: const EdgeInsets.all(16.0),
                    child: SingleChildScrollView(

```

```

child: Column(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    TextField(
      controller: _uController,
      decoration: InputDecoration(labelText: 'U: '),
      keyboardType: TextInputType.number,
    ),
    TextField(
      controller: _vController,
      decoration: InputDecoration(labelText: 'V: '),
      keyboardType: TextInputType.number,
    ),
    SizedBox(height: 20),
    Text('Двоичное представление u: $_binaryU'),
    SizedBox(height: 10),
    Text('Двоичное представление v: $_binaryV'),
    SizedBox(height: 20),
    Text('Время отработки метода ( Карацубы): $_karatsuba_time'),
    SizedBox(height: 10),
    Text('Время отработки обычное ( умножение): $_normal_time'),
    SizedBox(height: 25),
    Text('Результат умножения: $_result'),
    SizedBox(height: 10),
    ElevatedButton(
      onPressed: _calculate,
      child: Text('Преобразовать и умножить '),
    ),
  ],
),
),
),
),
);
}
}

```

3 Результаты

Результат выполнения программы:

Метод А.А. Карацубы

U:
813654256234567890980987654

V:
360876553456789875432679876745678

Двоичное представление u: 1010100001000010011111110110011100111100101100110000001110111011011000100110001100000110

Двоичное представление v: 1000111001010111001110000100010001011010111101010010010010100110110001010111001110010000001100101100111001110

Время отработки (метода Карацубы): 203 мс

Время отработки (обычное умножение): 519 мс

Результат умножения: 293628743695378646292356674012464780310918622301171815859412

[Преобразовать и умножить](#)

4 Выводы

В ходе выполнения лабораторной работы было реализовано dart-приложение, демонстрирующее работу алгоритма А.А. Карацубы и его сравнение с обычным перемножением.